```cpp
#ifndef REDNEURONAL_HPP
#define REDNEURONAL_HPP
#pragma once
#include "capaDensa.hpp"
#include "matrix.hpp"
#include <vector>
#include <functional>
#include "FuncionesPerdida.hpp"

using namespace std;

class RedNeuronal{
private:
    vector<capaDensa> capas_;
public:
    RedNeuronal(){
    }
    ~RedNeuronal(){
    }
    void agregarCapa(const capaDensa& capa){
        capas_.push_back(capa);
    }
    Matriz forward(const Matriz& entrada){
        Matriz actual = entrada;
        for(size_t i = 0; i < capas_.size();i++){
            actual = capas_[i].forward(actual);
        }
        return actual;
    }
    Matriz backward(Matriz gradianteInicial, double tasaDeAprendizaje, int t) {
        Matriz gradActual = gradianteInicial;
        // Recorremos las capas hacia atrás pasando la variable 't'
        for (int i = capas_.size() - 1; i >= 0; i--) {
            gradActual = capas_[i].backward(gradActual, tasaDeAprendizaje, t);
        }
        return gradActual;
    }

    void entrenar(const vector<Matriz>& entradas,
                  const vector<Matriz>& objetivos,
                  size_t epocas,
                  double tasaDeAprendizaje,
                  std::function<Matriz(const Matriz&, const Matriz&)> funcionPerdida =
gradienteMSE)
    {
        int t = 1; // Adam requiere que el contador de tiempo inicie en 1

        for (size_t e = 0; e < epocas; e++) {
            for (size_t k = 0; k < entradas.size(); k++) {

                // 1. Paso hacia adelante
                Matriz prediccion = forward(entradas[k]);

                // 2. Cálculo del gradiente inicial con la función inyectada
                Matriz gradiente = funcionPerdida(prediccion, objetivos[k]);

                // 3. Propagación hacia atrás enviando el tiempo 't'
                backward(gradiente, tasaDeAprendizaje, t);

                t++; // Se incrementa por cada dato procesado
            }
        }
    }
    void guardarModelo(const string& rutaArchivo) const {
        ofstream archivo(rutaArchivo);
        if (!archivo.is_open()) {
```

```
            cerr << "Error: No se pudo crear el archivo " << rutaArchivo << "\n";
            return;
        }

    // 1. Guardamos la cantidad de capas para validar al cargar
        archivo << capas_.size() << "\n";

    // 2. Recorremos cada capa y llamamos a su método guardar()
        for (size_t i = 0; i < capas_.size(); i++) {
            capas_[i].guardar(archivo);
        }

        archivo.close();
        cout << "Modelo guardado exitosamente en: " << rutaArchivo << "\n";
    }

    void cargarModelo(const string& rutaArchivo) {
        ifstream archivo(rutaArchivo);
        if (!archivo.is_open()) {
            cerr << "Error: No se pudo abrir el archivo " << rutaArchivo << "\n";
            return;
        }

        // 1. Leemos el número de capas del archivo
        size_t numCapas;
        archivo >> numCapas;

        // 2. Validamos que el archivo corresponda con la estructura de la red
        if (numCapas != capas_.size()) {
            cerr << "Error: La arquitectura del archivo no coincide con la red.\n";
            return;
        }

        // 3. Recorremos cada capa y llamamos a su método cargar()
        for (size_t i = 0; i < capas_.size(); i++) {
            capas_[i].cargar(archivo);
        }

        archivo.close();
        cout << "Modelo cargado exitosamente desde: " << rutaArchivo << "\n";
    }

};


#endif // REDNEURONAL_HPP
```